HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Quiz # 1 Solution

QUESTION 1: (20 marks)

You are given an input string called ***str1*** containing 'n' characters WITHOUT any blank spaces, where 'n' is at least 2. You are also given a queue called ***QueueChars*** of size 'n', which is initially empty. The program combines the repeating characters in the string in the same order as they appear in the given string, ***str1*** by following the steps as given below. Assume 'str1' is correctly given to you with the allowed length, so you may use it directly in your algorithm, without having to do any error checking or taking input. However, do declare it in the algorithm to define what it is, including the queues that you may use for your program.

DO NOT TRY TO ACHIEVE THE TASK IN ANY WAY OTHER THAN THE FOLLOWING STEPS:

1. It reads each character of string, ***str1*** and adds them to the queue, ***QueueChars*** one at a time, such that the first letter of the string appears at the front of the queue.
2. Once all letters are in the queue, you CANNOT use ***str1*** again anywhere in the program. If it is being used for any reason – simple or complex, the remaining algorithm will NOT be graded
3. Using the queue of letters, i.e. ***QueueChars***, you should generate a new queue, called ***QueueRepeat*** that will contain all the letters combined together. If there is one such letter in the queue, it will appear once in the same order as it was defined in the original queue.

For example,

- Assume ***str1*** = "**bippitybop"**, so as a result of first task, ***QueueChars*** = ['b', 'i', 'p', 'p', 'i', 't', 'y', 'b', 'o', 'p']
- Once ***QueueChars*** is filled, the resulting queue, ***QueueRepeat*** will contain all the letters combined together in the same order of their appearance. That is, for this example, ***QueueRepeat*** = ['b', 'b', 'i', 'i', 'p', 'p', 'p', 't', 'y', 'o']

You must implement the above tasks by writing an algorithm for each part.

Restrictions:

- Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized string.
- Assume the length of string is 'n', where n > 1, so there will always be at least two letters in any given string, resulting in Queues of size at least 2
- Assume None represents empty slots in all the data structures used for this question
- You are **NOT** allowed to use **any** additional **ADT**, including ListADT
- **DO NOT USE ANY OTHER DATA STRUCTURES INCLUDING ListADT/Python lists apart from the ones permitted to use**
- **REMEMBER: Stacks and Queues are non-iterable and non-traversing data structures, so you CANNOT and MUST NOT do indexing within the data structures to access the elements**

SOLUTION:

#Required variable declarations and initialization

- str1 – string input without spaces containing 'n' characters
- n – an integer representing the length of the string as well as the size of the queue, i.e., QueueChars
- QueueChars = [None] * n
- QueueRepeat = [None] * n

#Adding all the characters from the given string into the input queue, i.e., QueueChars

- for each character in string, do the following:
    - enqueue character in QueueChars

- i = 0 #index variable to run the loop for repeated characters
- lenQueueChars = n – 1 #integer variable to run the loop for existing number of characters
- character1 = dequeue from QueueChars
- enqueue character1 in QueueRepeat

#For the number of characters present in QueueChars at any point in time, we need to take the repetitions of all #the characters in the order they appear, one at a time from QueueChars, and add it to the output queue, i.e., #QueueRepeat. This loop will run for all the characters that are being repeated, and will still work in case of any #string without repeated characters

- while i <= lenQueueChars, do the following:
    - for lenQueueChars times, do the following:
        - character2 = dequeue from QueueChars
        - if character1 == character2:
            - enqueue character2 in QueueRepeat
        - else:
            - enqueue character2 in QueueChars
    - character1 = dequeue from QueueChars
    - enqueue character1 in QueueRepeat
    - lenQueueChars = number of elements in QueueChars
    - i += 1

#The following loop ensures that any characters that have been left behind in QueueChars, which are most likely #the ones that are not repeated, they are also added to the output queue, i.e., QueueRepeat with single instance

- while QueueChars is not empty, do the following:
    - character1 = dequeue from QueueChars
    - enqueue character1 in QueueRepeat

QUESTION 2: (5 marks)

Assume non-circular ListADT of size 4, with empty slots represented as None values in ADT. Fill the given table for the specified ListADT functions

SOLUTION:

Operation	Return Value	ListADT Contents
L.is_full()	False	[None, None, None, None]
L.insert_last('A')	–	[A, None, None, None]
L.insert_last('B')	–	[A, B, None, None]
length(L)	2	[A, B, None, None]
L.insert_first('C')	–	[C, A, B, None]
L.remove_last()	B	[C, A, None, None]
L.insert_last('T')	–	[C, A, T, None]
L.is_full()	False	[C, A, T, None]
L.remove_first()	C	[A, T, None, None]
length(L)	2	[A, T, None, None]
L.remove_first()	A	[T, None, None, None]
L.remove_last()	T	[None, None, None, None]
L.is_empty()	True	[None, None, None, None]